

American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies

CSE 4101 / [Course Code]

PROJECT REPORT

Project Title:

Travel Guide

Semester: [Spring 2025–2026]

Submission Date: [May-18-2026]

Section: [L]

Submitted To:

[WAHIDUL ALAM RIYAD]

[CSE]

American International University-Bangladesh

Group Information

Group Number: 08

Project Title: Travel Guide

Student Information:

Full Name: Tahsin Ahmed Fahim

Student ID: 23-52735-2

Program: CSE

Email: tahsinahmedfahim01611@gmail.com

Signature: _____

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.		Section	
Member 1 ID			

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			Feature Implementation (0-3)		Completeness (0-3)
Basic Web Security	UI (HTML/CSS)	Feature Completeness	DB	Auth (Session/cookie)	MVC
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			Collaborative Teamwork (0-3)		Communication/Promptness (0-3)
JS validation	PHP Validation	Ajax/JSON	Git Contribution		VIVA
Expected Outcome (0-2)		CO3 Total	CO4 Total		Project Total

Evaluated By: _____ **Date:** _____

Course Teacher Signature: _____

Table of Contents

1. Group Information	2
2. Marks Sheet	3
3. Table of Contents	4
4. Chapter 1: Project Overview	5
4.1 Project Description	5
4.1 Technologies Used	5
4.3 System Architecture / MVC Diagram	6
4.4 Database Schema	6
5. Chapter 2: Git Contribution & Task Allocation	7
5.1 GitHub Repository Link	7
5.2 Git Contribution Screenshot	8
6. Chapter 3: Features, Source Code & Screenshots	9
7. Chapter 5: Conclusion & Future Work	20

Chapter 1: Project Overview

1.1 Project Description

Travel Guide is a web-based application designed to help visitors from different countries discover and explore travel destinations worldwide. The platform solves the problem of finding reliable, curated travel information by connecting three types of users in a structured workflow.

Scouts collect and submit detailed information about visiting places including costs, travel mediums, short histories, and country representations. Administrators review and approve these submissions before they become publicly visible. General Users can then browse approved destinations, save favorites to their personal wishlist, get probable cost estimates, and post comments.

My role in this project (Task 1) focuses on the complete user authentication system including registration for all three roles, secure login with Remember Me functionality, profile management with picture upload, the home page displaying approved destinations, and the wishlist feature allowing verified General Users to save and remove travel destinations using AJAX without page reload.

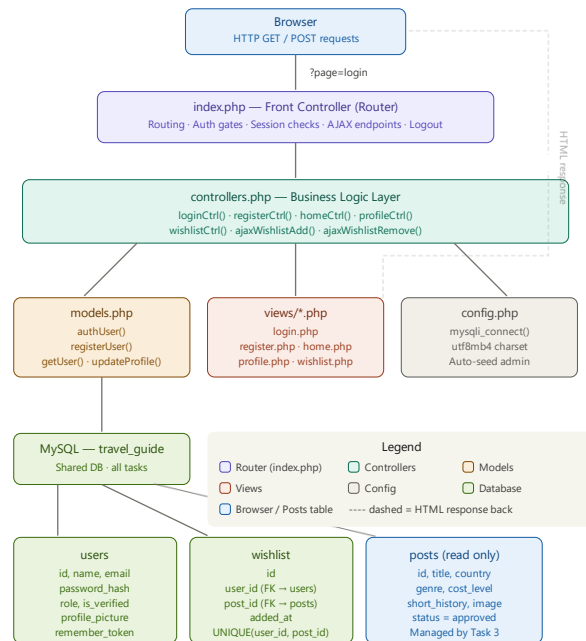
The target users are travel enthusiasts seeking destination inspiration, content scouts who contribute place information, and administrators who moderate and publish content to maintain quality.

1.2 Technologies Used

List all technologies, frameworks, and tools used in this project.

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3, Bootstrap	UI & Styling
	[e.g., JavaScript / jQuery]	[e.g., Client-side validation, AJAX]
Backend	[e.g., PHP]	[e.g., Server-side logic]
Database	[e.g., MySQL]	[e.g., Data storage]
Auth	[e.g., PHP Sessions / Cookies]	[e.g., Login management]
Version Control	Git / GitHub	Collaboration & versioning
Server	[e.g., XAMPP / Apache]	[e.g., Local development]

1.3 System Architecture / MVC Diagram



1.4 Database Schema

Table	Field Name	Data Type	Description
users	id	INT AUTO_INCREMENT	Primary Key
users	name	VARCHAR(100)	Full name
users	email	VARCHAR(150) UNIQUE	Login email
users	password_hash	VARCHAR(255)	Bcrypt hashed password
users	role	ENUM(admin,scout,user)	User role
users	is_verified	TINYINT(1)	Admin verification status
users	profile_picture	VARCHAR(255)	Path to uploaded image
users	remember_token	VARCHAR(255)	Hashed Remember Me token
users	created_at	DATETIME	Registration timestamp
wishlist	id	INT AUTO_INCREMENT	Primary Key
wishlist	user_id	INT (FK)	References users.id
wishlist	post_id	INT (FK)	References posts.id
wishlist	added_at	DATETIME	When post was saved
posts	id	INT AUTO_INCREMENT	Primary Key (read only)
posts	scout_id	INT (FK)	References users.id
posts	title	VARCHAR(200)	Place title

posts	short_history	TEXT	Description of the place
posts	country	VARCHAR(100)	Country name
posts	genre	VARCHAR(50)	beach/mountain/city/historical
posts	cost_level	ENUM(low,medium,high)	Cost category
posts	travel_medium_info	VARCHAR(200)	Flight/train/bus info
posts	image	VARCHAR(255)	Path to post image
posts	status	ENUM(approved)	Only approved posts shown — managed by Task 3
posts	created_at	DATETIME	Approval timestamp

Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

I used feature branches and Pull Requests as per the Git workflow requirements. My commits were made on the branch feature/Task1-23-52735-2 and merged into main via a Pull Request.

Repository URL: <https://github.com/Shihabusa/travel-guide-group08>

2.2 Git Contribution Screenshot

2.2 Git Contribution Screenshot

Merge pull request #3 from Shihabusa/feature/Task1-23-52735-2 CodeFahim authored 1 hour ago	Verified 65e927f	
refactor: move task1 files into task1 subfolder CodeFahim committed 1 hour ago	55f2972	
Merge pull request #2 from Shihabusa/feature/Task1-23-52735-2 CodeFahim authored 3 hours ago	Verified d914a66	
feat: add upload directories for profile pictures CodeFahim committed 3 hours ago	6c2fc18	
Merge pull request #1 from Shihabusa/feature/Task1-23-52735-2 CodeFahim authored 8 hours ago	Verified 97039f	
feat: add wishlist page view CodeFahim committed 8 hours ago	e3e3c53	
feat: add profile page view CodeFahim committed 8 hours ago	5103293	
feat: add home page view CodeFahim committed 8 hours ago	f9a9d9f	
feat: add register view CodeFahim committed 8 hours ago	47d8fbf	
feat: add login view CodeFahim committed 8 hours ago	4970f1d	
style: add travel guide theme and styling CodeFahim committed 8 hours ago	ad4a37d	
feat: add front controller and routing CodeFahim committed 8 hours ago	e5e91b3	
feat: add login, register, profile and wishlist controllers CodeFahim committed 8 hours ago	3ea8aa2	
feat: add auth, profile and wishlist models CodeFahim committed 8 hours ago	2ea97d3	
feat: add database connection config CodeFahim committed 8 hours ago	4a07660	
feat: add shared database schema CodeFahim committed 8 hours ago	457eca0	

This branch is 15 commits behind main.			Contribute
Name	Last commit message	Last commit date	
...			
public/uploads/profiles	refactor: move task1 files into task1 subfolder	1 hour ago	
views	refactor: move task1 files into task1 subfolder	1 hour ago	
config.php	refactor: move task1 files into task1 subfolder	1 hour ago	
controllers.php	refactor: move task1 files into task1 subfolder	1 hour ago	
database.sql	refactor: move task1 files into task1 subfolder	1 hour ago	
index.php	refactor: move task1 files into task1 subfolder	1 hour ago	
models.php	refactor: move task1 files into task1 subfolder	1 hour ago	
style.css	refactor: move task1 files into task1 subfolder	1 hour ago	

Chapter 3: Features, Source Code & Screenshots

For each feature implemented in the project, provide the following: a description, the relevant source code with explanation, and screenshot(s) of the UI.

Duplicate the feature section (Feature 1 block) as many times as needed — one block per feature.

Feature 1: [User Registration]

Description

The registration system allows users to create accounts with three roles: Admin, Scout, and General User. It collects name, email, password, and role. All passwords are hashed using PHP's `password_hash()` function. New accounts are set to `is_verified = 0` by default, requiring admin approval. After successful registration, users are redirected to the login page with a flash message.

Relevant Source Code

File: `/controllers.php` — `registerCtrl()` function

```
function registerCtrl($conn) {
    $error    = '';
    $success  = '';
    $old      = ['name' => '', 'email' => '', 'role' => 'user'];

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $name      = trim($_POST['name'] ?? '');
        $email     = trim($_POST['email'] ?? '');
        $password  = $_POST['password'] ?? '';
        $confirm   = $_POST['confirm_password'] ?? '';
        $role      = $_POST['role'] ?? 'user';
        $old       = compact('name', 'email', 'role');

        // Server side validation
        if ($name === '' || $email === '' || $password === '' || $confirm ===
        '') {
            $error = 'All fields are required.';
        } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            $error = 'Please enter a valid email address.';
        } elseif (strlen($password) < 8) {
            $error = 'Password must be at least 8 characters.';
        } elseif ($password !== $confirm) {
            $error = 'Passwords do not match.';
        } elseif (!in_array($role, ['admin', 'scout', 'user'])) {
            $error = 'Invalid role selected.';
        } elseif (emailExists($conn, $email)) {
            $error = 'Email already exists.';
        }
    }

    if ($error === '') {
        $success = 'Registration successful. You will be redirected to the login page.';
    }
}
```

```

        $error = 'This email is already registered.';
    } else {
        if (registerUser($conn, $name, $email, $password, $role)) {
            header('Location: index.php?page=login&msg=registered');
            exit;
        } else {
            $error = 'Registration failed. Please try again.';
        }
    }
}

require 'views/register.php';
}

```

File: /models.php — registerUser() function

```

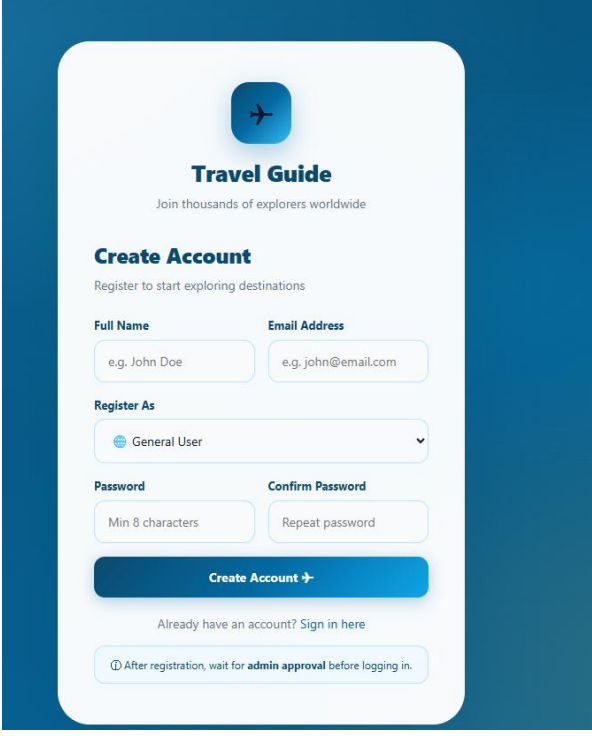
function registerUser($conn, $name, $email, $password, $role) {
    $hash = password_hash($password, PASSWORD_DEFAULT);
    $stmt = mysqli_prepare($conn,
        "INSERT INTO users (name, email, password_hash, role, is_verified)
        VALUES (?, ?, ?, ?, 0)");
    mysqli_stmt_bind_param($stmt, 'sssss', $name, $email, $hash, $role);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

```

Code Explanation

- trim() removes whitespace from user inputs to prevent empty string attacks.
- filter_var() with FILTER_VALIDATE_EMAIL validates email format server-side.
- strlen(\$password) < 8 enforces minimum password length of 8 characters.
- emailExists() checks the database for duplicate emails using prepared statements.
- password_hash() with PASSWORD_DEFAULT uses bcrypt algorithm for secure password storage.
- is_verified = 0 is hardcoded so all new users require admin approval.
- Redirect to login with flash message msg=registered as required by PDF.

Screenshot(s)



Travel Guide
Join thousands of explorers worldwide

Create Account
Register to start exploring destinations

Full Name
e.g. John Doe

Email Address
e.g. john@email.com

Register As
General User

Password
Min 8 characters

Confirm Password
Repeat password

Create Account

Already have an account? Sign in here

After registration, wait for **admin approval** before logging in.

Feature 2: [Login & Remember Me]

Description

The login system authenticates users by email and password. It creates PHP session variables `$_SESSION['user_id']`, `$_SESSION['name']`, and `$_SESSION['role']`. The Remember Me feature stores a secure random token (hashed with SHA-256) in the database and sets a 30-day cookie. On subsequent visits, PHP automatically reinstates the session from the cookie.

Relevant Source Code

File: /controllers.php — loginCtrl() Remember Me section

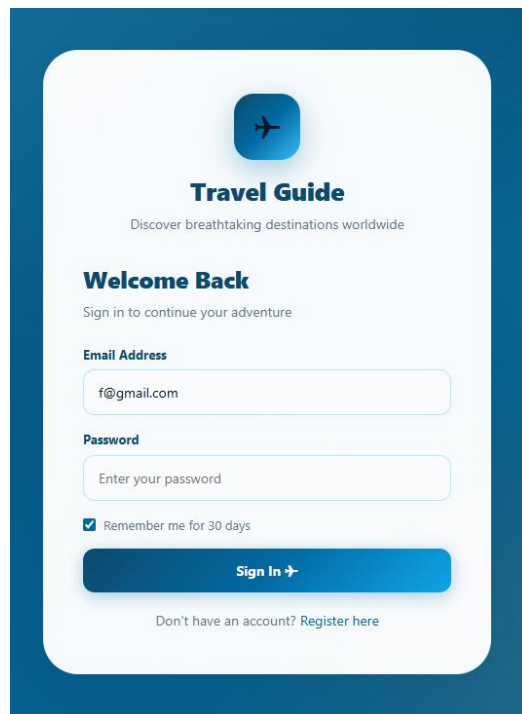
```
if ($remember) {
    $token = bin2hex(random_bytes(32));
    saveRememberToken($conn, $user['id'], $token);
    setcookie('remember_token', $token, time() + 86400 * 30, '/',
'', false, false);
    setcookie('remember_email', $email, time() + 86400 * 30, '/',
'', false, false);
} else {
    setcookie('remember_token', '', time() - 3600, '/', '',
false, false);
    setcookie('remember_email', '', time() - 3600, '/', '',
false, false);
}
```

```
        header('Location: index.php?page=home');  
        exit;  
    }  
    $error = 'Invalid email or password.';  
}  
}
```

Code Explanation

- `bin2hex(random_bytes(32))` generates a cryptographically secure 64-character token.
- `saveRememberToken()` hashes the token with SHA-256 before storing in DB — raw token never stored.
- Cookie expires in $86400 * 30$ seconds (30 days) as required.
- On next visit, cookie token is hashed and compared against DB hash for auto-login.
- Session stores `user_id`, `name`, and `role` as required.

Screenshot(s)



Feature 3: [Profile Page]

Description

The profile page allows authenticated users to update their name, email, and profile picture. It also supports password change by requiring the current password for verification. A success banner is displayed after each successful update. Profile pictures are validated for MIME type and size (max 2MB) before upload.

Relevant Source Code

File: /controllers.php — profileCtrl() image upload section

```
// Handle profile picture upload
    if (!empty($_FILES['profile_picture']['name'])) {
        $file      = $_FILES['profile_picture'];
        $allowed   = ['image/jpeg', 'image/png', 'image/gif',
'allowed. ';
        $maxSize   = 2 * 1024 * 1024; // 2MB

        if (!in_array($file['type'], $allowed)) {
            $error = 'Only JPG, PNG, GIF or WEBP images are

        } elseif ($file['size'] > $maxSize) {
            $error = 'Image must be under 2MB.';
        } else {
            $ext      = pathinfo($file['name'], PATHINFO_EXTENSION);
            $fname     = 'user_' . $userId . '_' . time() . '.' . $ext;
            $dest      = 'public/uploads/profiles/' . $fname;
            if (move_uploaded_file($file['tmp_name'], $dest)) {
                $picture = $dest;
            } else {
                $error = 'Failed to upload image.';
            }
        }
    }
}
```

Code Explanation

- `in_array()` checks the MIME type against a whitelist to prevent malicious uploads.
- File size is checked against 2MB limit before moving to uploads directory.
- Unique filename is generated using user ID and timestamp to prevent collisions.
- `move_uploaded_file()` safely moves the temp file to `public/uploads/profiles/`.
- Password change requires current password verification using `password_verify()`.

Screenshot(s)

Feature 4: [Home Page]

Description

The home page shows different content based on user status. Non-registered users see a simple greeting with call-to-action buttons. Logged-in but unverified users see a pending approval message. Verified users see the latest approved travel destination posts with a hero banner and statistics bar.

Relevant Source Code

File: /controllers.php — homeCtrl()

```
function homeCtrl($conn) {
    $posts = [];
    if (isset($_SESSION['user']) && $_SESSION['user']['is_verified']) {
        $posts = getApprovedPosts($conn);
    }
    require 'views/home.php';
}
```

File: /models.php — getApprovedPosts() function

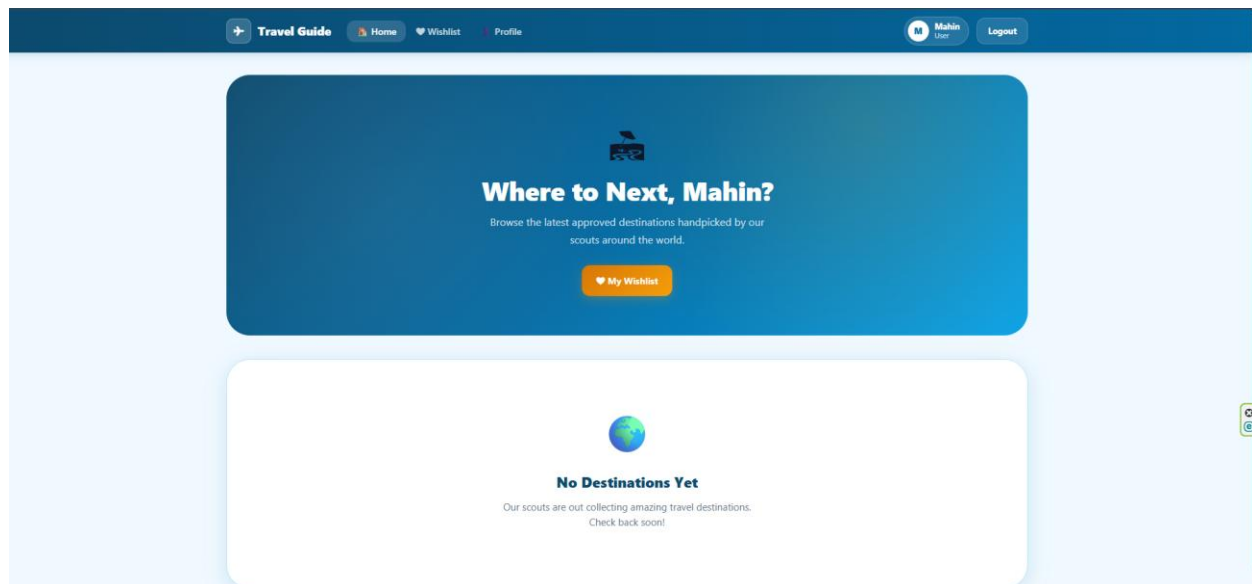
```
/* Home Page Posts */
```

```
function getApprovedPosts($conn) {
    $r = mysqli_query($conn,
        "SELECT id, title, country, genre, cost_level, short_history, image
        FROM posts WHERE status = 'approved'
        ORDER BY created_at DESC");
    return mysqli_fetch_all($r, MYSQLI_ASSOC);
}
```

Code Explanation

- homeCtrl() checks if user is logged in AND verified before fetching posts.
- getApprovedPosts() only returns posts with status = 'approved' ensuring content moderation.
- Posts are ordered by created_at DESC to show newest destinations first.
- The view handles three states: non-registered, pending, and verified users.

Screenshot(s)



Feature 5: [Wishlist (AJAX CRUD)]

Description

The Wishlist feature allows only verified General Users (role='user' and is_verified=1) to save and remove travel destinations. Add and Remove operations use AJAX with JSON responses, updating the UI without page reload. All database operations use prepared statements for SQL injection prevention.

Relevant Source Code

File: /controllers.php — ajaxWishlistAdd()

```
/* AJAX - Wishlist Add */
function ajaxWishlistAdd($conn) {
```

```

header('Content-Type: application/json');

if (!isset($_SESSION['user'])) {
    http_response_code(403);
    echo json_encode(['error' => 'Unauthorized']);
    exit;
}

$user = $_SESSION['user'];
if ($user['role'] !== 'user' || !$user['is_verified']) {
    http_response_code(403);
    echo json_encode(['error' => 'Only verified general users can use the
wishlist.']);
    exit;
}

$data = json_decode(file_get_contents('php://input'), true);
$postId = intval($data['post_id'] ?? 0);

if ($postId <= 0) {
    http_response_code(400);
    echo json_encode(['error' => 'Invalid post.']);
    exit;
}

if (isInWishlist($conn, $user['id'], $postId)) {
    echo json_encode(['success' => false, 'message' => 'Already in
wishlist.']);
    exit;
}

if (addToWishlist($conn, $user['id'], $postId)) {
    echo json_encode(['success' => true, 'message' => 'Added to wishlist!']);
} else {
    http_response_code(500);
    echo json_encode(['error' => 'Failed to add to wishlist.']);
}
exit;
}

```

File: /views/home.php — AJAX fetch call

```

fetch('index.php?page=ajax&type=wishlist_add', {
    method: 'POST',
    credentials: 'same-origin',

```



```

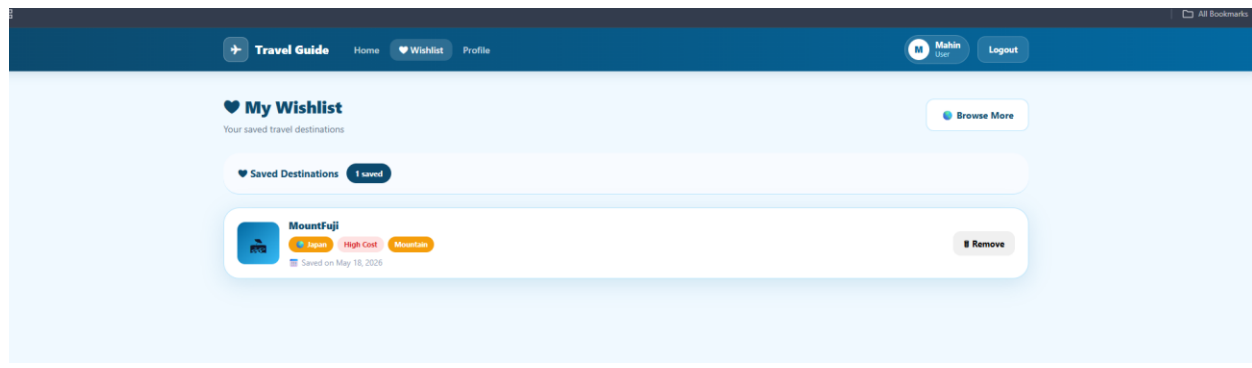
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ post_id: parseInt(postId) })
    })
    .then(function (r) { return r.json(); })
    .then(function (data) {
        if (data.success) {
            self.textContent = '&#10004; Saved!';
            self.style.background = '#16a34a';
        } else {
            self.textContent = '&#10084; Already Saved';
            self.disabled = false;
        }
    })
    .catch(function () {
        self.textContent = '&#10084; Add to Wishlist';
        self.disabled = false;
    });
});

```

Code Explanation

- AJAX endpoint checks role = 'user' AND is_verified = 1 before allowing wishlist access.
- Content-Type: application/json header ensures proper JSON response format.
- http_response_code(403) returns proper HTTP status for unauthorized access.
- isInWishlist() prevents duplicate entries before adding.
- INSERT IGNORE in addToWishlist() provides database-level duplicate protection.
- Client-side fetch() updates UI without page reload for better user experience.
- Remove uses same AJAX pattern with ajaxWishlistRemove() endpoint.

Screenshot(s)



Feature 6: [JavaScript Validation]

Description

All forms include client-side JavaScript validation for immediate user feedback. Registration validates email format, password length (min 8 chars), password match, and required fields. Profile form validates name, email, and password fields. Error messages appear inline below each field.

Relevant Source Code

File: /views/register.php — JS validation

```
/*Email format validation*/
function validateEmail(val) {
    return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(val);
}

function showErr(el, msg) {
    el.textContent = msg;
    el.style.display = 'block';
}

function clearErr(el) {
    el.textContent = '';
    el.style.display = 'none';
}

/* Live password validation on input */

pass.addEventListener('input', function () {
    if (pass.value === '') {
        showErr(passErr, 'Password is required.');
```

```
    } else if (pass.value.length < 8) {
        showErr(passErr, 'Password must be at least 8 characters.');
```

```
    } else {
        clearErr(passErr);
    }
    if (confirm.value !== '' && pass.value !== confirm.value) {
        showErr(confirmErr, 'Passwords do not match.');
```

```
    } else {
        clearErr(confirmErr);
    }
});

/* Submit validation */
form.addEventListener('submit', function (e) {
    var valid = true;
```

```
        if (name.value.trim() === '') {
            showError(nameErr, 'Full name is required.');
```

```
        }; valid = false;
    }
    if (email.value.trim() === '') {
        showError(emailErr, 'Email is required.');
```

```
    }; valid = false;
    } else if (!validateEmail(email.value.trim())) {
        showError(emailErr, 'Please enter a valid email address.');
```

```
    }; valid =
false;
    }
    if (pass.value === '') {
        showError(passErr, 'Password is required.');
```

```
    }; valid = false;
    } else if (pass.value.length < 8) {
        showError(passErr, 'Password must be at least 8 characters.');
```

```
    }; valid =
false;
    }
    if (confirm.value === '') {
        showError(confirmErr, 'Please confirm your password.');
```

```
    }; valid = false;
    } else if (confirm.value !== pass.value) {
        showError(confirmErr, 'Passwords do not match.');
```

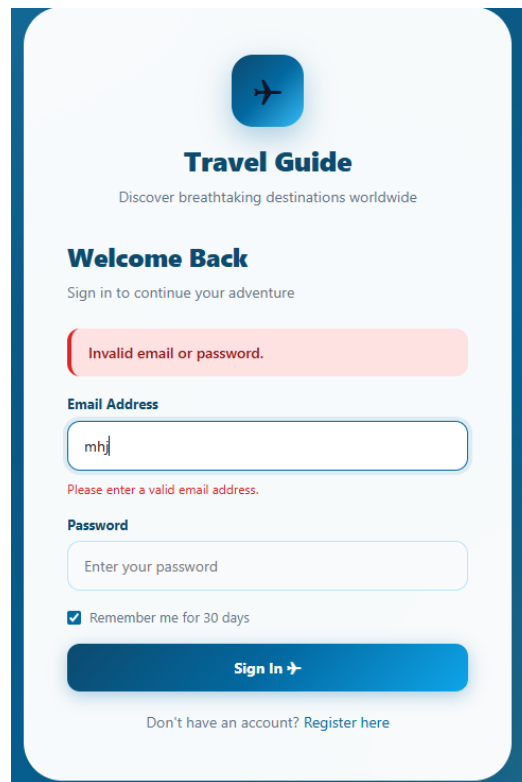
```
    }; valid = false;
    }

    if (!valid) e.preventDefault();
});
})();
```

Code Explanation

- Regex `/^[^s@]+@[^s@]+\.[^s@]+$/` validates email format on every keystroke.
- Live validation on 'input' event gives immediate feedback as user types.
- `e.preventDefault()` stops form submission if any field is invalid.
- `showErr()` and `clearErr()` functions manage inline error message display.

Screenshot(s)



Chapter 5: Conclusion & Future Work

5.1 Summary

Task 1 of the Travel Guide project covered the complete user authentication system, profile management, home page, and wishlist functionality for General Users. All 10 grading criteria were successfully implemented including Basic Web Security, UI/CSS, Feature Completeness, Database, Auth (Session/Cookie), MVC, JS Validation, PHP Validation, AJAX/JSON, and Git Contribution.

The system uses procedural PHP with mysqli prepared statements throughout, ensuring SQL injection prevention. Passwords are hashed using bcrypt via `password_hash()`. The Remember Me feature uses cryptographically secure tokens. All forms have both client-side and server-side validation.

5.2 Challenges Faced

- Remember Me Cookie Issue: Cookies were not being set on localhost due to PHP.ini session.cookie_samesite setting. Fixed by setting session.cookie_samesite=Lax in php.ini.
- Session Redirect Loop: Task 2 scout pages caused infinite redirect loops when a non-scout user tried to access them. Fixed by properly checking role and is_verified before redirecting.
- Profile Picture Upload Path: The public/uploads/profiles/ directory did not exist causing upload failures. Fixed by manually creating the directory structure.
- Git Merge Conflicts: Moving files into task1/ subfolder caused commit history to show refactor commit for all files. Resolved by checking feature branch history instead of main.

5.3 Future Improvements

- Email Verification: Send automated email verification link instead of requiring manual admin approval for new accounts.
- OAuth Login: Add Google and Facebook login options using OAuth 2.0 for easier registration.
- Wishlist Categories: Allow users to organize wishlist items into custom categories or travel plans.
- Post Rating System: Allow verified users to rate and review approved destinations with star ratings.
- Advanced Search: Add search and filter functionality to the home page for browsing destinations by country, genre, and cost level.
- Mobile App: Develop a companion mobile application using React Native that connects to the same backend API.

References

- [1] PHP Manual. password_hash() — Secure password hashing function. Available at: <https://www.php.net/manual/en/function.password-hash.php>
- [2] PHP Manual. mysqli_prepare() — Prepared statements for SQL injection prevention. Available at: <https://www.php.net/manual/en/mysqli.prepare.php>
- [3] Mozilla MDN Web Docs. Fetch API — AJAX requests in JavaScript. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [4] PHP Manual. session_start() — Session management in PHP. Available at: <https://www.php.net/manual/en/function.session-start.php>